

<h1 id="Week-4,-Day-3">Week 4, Day 3</h1> <h2 id="Feature-Engineering,-Cross-Validation-&-Model-Comparison">Feature Engineering, Cross Validation & Model Comparison</h2> <p>Continuing from Day 2. Same train_df / dev_df / test_df split (RANDOM_STATE=42), same hold out test set, same primary metric (F1, with ROC AUC as secondary).</p> <p>Everything in this notebook happens on train_df only. The hold out test_df stays untouched until a final report, exactly like Day 1 and Day 2 established.</p>

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
    ConfusionMatrixDisplay, classification_report
)

RANDOM_STATE = 42 # fixed seed -> same split every time we re-run this notebook
pd.set_option('display.max_columns', 50)
sns.set_theme(style="whitegrid")
```

<h2 id="Data-Load-&-Quick-EDA">Data Load & Quick EDA</h2> <p>**Missing values:** this dataset marks missing entries with the string '?' instead of a proper NaN. We convert those to real NaN so pandas/sklearn treat them as missing (and so .isna() actually finds them).</p>

```
In [4]: # Column names, per the official UCI documentation for this dataset
column_names = [
    "age", "workclass", "fnlwgt", "education", "education-num",
    "marital-status", "occupation", "relationship", "race", "sex",
    "capital-gain", "capital-loss", "hours-per-week", "native-country", "income"
]

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/adult-all.csv"
df = pd.read_csv(url, header=None, names=column_names, na_values="?", skipinitia

print("Shape:", df.shape)
df.head()
```

Shape: (48842, 15)

```
Out[4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>age</th> <th>workclass</th> <th>fnlwgt</th> <th>education</th>
<th>education-num</th> <th>marital-status</th> <th>occupation</th>
<th>relationship</th> <th>race</th> <th>sex</th> <th>capital-gain</th>
<th>capital-loss</th> <th>hours-per-week</th> <th>native-country</th>
<th>income</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174
50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0
38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0
53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0
28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0



```
In [5]: # Converts he target to binary 0/1.
# income comes in as '<=50K' or '>50K' (sometimes with a trailing period in some
df["income"] = df["income"].str.strip().str.rstrip(".")
df["target"] = (df["income"] == ">50K").astype(int)

df[["income", "target"]].drop_duplicates()
```

```
Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>income</th> <th>target</th> </thead> <tbody> <th>0</th>
<th>7</th> </tbody> <tbody></tbody>
```

```
<=50K  0
>50K   1
```

```
In [6]: # Column types and missingness
print(df.dtypes)
print()
missing = df.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
print("Columns with missing values:")
print(missing)
print()
print(f"Total missing cells: {df.isna().sum().sum()} "
      f"({df.isna().sum().sum() / df.size:.2%} of all cells)")
```

```

age                int64
workclass          str
fnlwgt            int64
education          str
education-num      int64
marital-status     str
occupation         str
relationship       str
race              str
sex               str
capital-gain       int64
capital-loss       int64
hours-per-week     int64
native-country     str
income            str
target            int64
dtype: object

```

Columns with missing values:

```

occupation        2809
workclass         2799
native-country     857
dtype: int64

```

Total missing cells: 6465 (0.83% of all cells)

```

In [7]: # Numeric summaries
numeric_cols = ["age", "education-num", "capital-gain", "capital-loss", "hours-p
df[numeric_cols].describe().T

```

```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%
</th> <th>50%</th> <th>75%</th> <th>max</th> </thead> <tbody> <th>age</th>
<th>education-num</th> <th>capital-gain</th> <th>capital-loss</th> <th>hours-per-
week</th> </tbody> <tbody></tbody>

```

48842.0	38.643585	13.710510	17.0	28.0	37.0	48.0	90.0
48842.0	10.078089	2.570973	1.0	9.0	10.0	12.0	16.0
48842.0	1079.067626	7452.019058	0.0	0.0	0.0	0.0	99999.0
48842.0	87.502314	403.004552	0.0	0.0	0.0	0.0	4356.0
48842.0	40.422382	12.391444	1.0	40.0	40.0	45.0	99.0

```

In [8]: # Class base rate -- how many people actually earn >50K?
base_rate = df["target"].mean()
class_counts = df["target"].value_counts().rename({0: "<=50K", 1: ">50K"})
print(class_counts)
print(f"\nBase rate (positive class share): {base_rate:.2%}")

```

```

target
<=50K    37155
>50K     11687
Name: count, dtype: int64

```

Base rate (positive class share): 23.93%

This base rate matters a lot: it's our "do-nothing" reference point. If ~24% of people earn >50K, then a model that *always* guesses "<=50K" is already approximately 76% accurate which is why accuracy alone is a misleading metric for this problem (more on this in Task 4).

Create Reproducible Splits... Continuing from where we left off yesterday

Why a hold-out test set at all? If we tune our model and pick our final approach by repeatedly checking performance on the *same* data we'll report our final number on, we're effectively "peeking at the answer key" we will unconsciously choose whatever settings happen to work best on that particular data, which **overstates** how well the model will do on genuinely new people. This is called **data leakage / overfitting to the test set**.

The fix: split off a hold-out test set right now, before any modeling, and don't look at it again until the very end when we report final numbers. Everything else (EDA, baseline tuning, feature engineering, model comparison) happens only on the training/dev data.

- We use `stratify=target` so the hold-out set has the same ~24%/76% class balance as the full dataset otherwise a random split could accidentally end up with too few positive examples to evaluate reliably.
- We fix `random_state=42` so that every time this notebook is re-run, we get the *exact same* split

```
In [9]: # 80% train+dev / 20% hold-out test, stratified on the target
train_dev_df, test_df = train_test_split(
    df, test_size=0.20, stratify=df["target"], random_state=RANDOM_STATE
)

# Optional further split: carve a dev set out of the remaining 80%
# (10% of the ORIGINAL data -> 12.5% of train_dev_df, since train_dev_df is 80%
train_dev_df, dev_df = train_test_split(
    train_dev_df, test_size=0.125, stratify=train_dev_df["target"], random_state=
)

print(f"Train size: {len(train_df)} ({len(train_df)/len(df):.1%} of full data)")
print(f"Dev size: {len(dev_df)} ({len(dev_df)/len(df):.1%} of full data)")
print(f"Test size: {len(test_df)} ({len(test_df)/len(df):.1%} of full data)")
print()
print("Class balance check (share of positives in each split):")
for name, part in [("Full data", df), ("Train", train_df), ("Dev", dev_df), ("Test", test_df)]:
    print(f" {name:18s}: {part['target'].mean():.2%}")
```

```
Train size: 34188 (70.0% of full data)
Dev size: 4885 (10.0% of full data)
Test size: 9769 (20.0% of full data)
```

```
Class balance check (share of positives in each split):
Full data      : 23.93%
Train          : 23.93%
Dev            : 23.93%
Test (hold-out) : 23.93%
```

Why this matters: A hold-out test set gives us an honest, one-time estimate of how the model will perform on people it has never seen which is the real-world use case. If we

use the test set during hyperparameter tuning (e.g. picking the best `max_depth` by checking test accuracy), we end up implicitly fitting the model to the test set's quirks, so our reported performance becomes optimistic and won't hold up in production. That's why we tune using the **train/dev** split and touch the **test** set only once, at the very end, for final reporting.

Implement Simple Baselines

Before building anything fancy, we build baselines: deliberately simple models that tell us the minimum bar any real model must clear to be worth using. If a complex model can't beat these, it's not adding value.

Baseline 1: Majority-class predictor. Always predicts the most common class ($\leq 50K$). This tells us what "doing nothing clever" looks like.

Baseline 2: Single-feature rule. We'll use `education-num >= 13` (13 = Bachelor's degree or higher in this dataset's encoding) OR `capital-gain > 0` as a simple "predict $> 50K$ " rule. *Justification:* both education level and capital gains are strongly, intuitively tied to income since more education tends to mean higher-paying work, and having any capital gains at all usually implies enough wealth to invest. This gives us a baseline that's simple but not naive.

```
In [10]: # Baseline 1: Majority-class predictor
majority_class = train_df["target"].mode()[0]
print(f"Majority class in training data: {majority_class} "
      f"({'<=50K' if majority_class == 0 else '>50K'})")

y_test = test_df["target"].values
y_pred_majority = np.full_like(y_test, fill_value=majority_class)
# "Probability" for a constant predictor -- use the class itself as a degenerate
y_score_majority = y_pred_majority.astype(float)
```

Majority class in training data: 0 ($\leq 50K$)

```
In [11]: # Baseline 2: Single-feature rule
# Rule: predict >50K if education-num >= 13 OR capital-gain > 0
def rule_predict(frame):
    return ((frame["education-num"] >= 13) | (frame["capital-gain"] > 0)).astype(float)

y_pred_rule = rule_predict(test_df).values
# A simple continuous "score" version for ROC/PR AUC: combine both signals into
y_score_rule = (
    (test_df["education-num"] / test_df["education-num"].max()) * 0.5
    + (test_df["capital-gain"] > 0).astype(float) * 0.5
).values
```

```
In [12]: def evaluate(y_true, y_pred, y_score, name):
    return {
        "model": name,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, y_score),
        "pr_auc": average_precision_score(y_true, y_score),
    }

results = [
    evaluate(y_test, y_pred_majority, y_score_majority, "Majority-class"),
```

```

    evaluate(y_test, y_pred_rule, y_score_rule, "Rule-based (education-num>=13 OR capital-gain>0)
]
results_df = pd.DataFrame(results).set_index("model").round(4)
results_df

```

```

Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>accuracy</th> <th>precision</th> <th>recall</th> <th>f1</th>
<th>roc_auc</th> <th>pr_auc</th> <th>model</th> <th></th> <th></th> <th>
</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Majority-class</th>
<th>Rule-based (education-num>=13 OR capital-gain>0)</th> </tbody> <tbody>
</tbody>

0.7607  0.0000  0.0000  0.0000  0.5000  0.2393

0.7452  0.4746  0.6031  0.5312  0.7426  0.4946

```

```

In [13]: rule_f1 = results_df.loc["Rule-based (education-num>=13 OR capital-gain>0)", "f1"]
rule_auc = results_df.loc["Rule-based (education-num>=13 OR capital-gain>0)", "roc_auc"]
maj_f1 = results_df.loc["Majority-class", "f1"]
maj_acc = results_df.loc["Majority-class", "accuracy"]
print(rule_f1)
print(rule_auc)
print(maj_f1)
print(maj_acc)

```

```

0.5312
0.7426
0.0
0.7607

```

<p>**Primary metric for the rest of the week.** We will continue optimizing for **F1-score**, supported by **ROC AUC** as a secondary check. Our rule-based baseline currently sits at **F1 = 0.5312 / ROC AUC = 0.7426** on the hold-out test set, clearly beating the majority-class baseline's F1 = 0 (which has zero precision/recall on the positive class despite 0.7607 accuracy). F1 was chosen because it directly reflects our business goal -- balancing wasted outreach (false positives, hurting precision) against missed opportunities (false negatives, hurting recall) -- in a single number that is hard to game by predicting only one class, unlike accuracy.</p>

<h1 id="Day-2-starts-here">Day 2 starts here</h1>

```

In [14]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import RocCurveDisplay, PrecisionRecallDisplay

```

<h2 id="Task-1:-Preprocessing-Plan-&-Implementation">Task 1: Preprocessing Plan & Implementation</h2><p>**Numeric features** (reusing numeric_cols from Day 1's EDA): age, education-num, capital-gain, capital-loss, hours-per-week</p><p>**Categorical features:** workclass, marital-status, occupation, relationship, race, sex, native-country</p><p>**Dropped:** education (redundant string duplicate

of education-num), income/target handled separately as the label. '?' -> NaN was already done in Day 1's .pynb, so no extra cleanup needed here.

Why median imputation (numeric)? Day 1's histograms showed capital-gain, capital-loss, and fnlwt are heavily right-skewed with outliers. Mean imputation gets dragged toward those outliers; the median doesn't. *Alternatives considered:* mean (rejected — skew-sensitive); KNN/iterative imputation (rejected — more complex than justified here, and none of these numeric columns actually had missing values in this dataset anyway, so this is mostly a safety net).

Why OneHotEncoder (categorical)? None of our categoricals have a natural order, so ordinal encoding would invent a false ranking that misleads Logistic Regression's linear weights. `handle_unknown='ignore'` protects against category levels unseen at training time, which matters most for native-country (long tail of rare countries). *Alternatives considered:* target/mean encoding (rejected — leaks target info without careful cross-fitting); ordinal encoding (rejected — no true order in these columns).

Missing categoricals: imputed with the *most frequent* value per column — simple, pairs cleanly with one-hot. *Alternative:* an explicit "Missing" category — flagged for Day 3 (Day 1 fix #1), since that would let "didn't answer" be its own learnable signal instead of folding into the majority class.

```
In [15]: categorical_features = ["workclass", "marital-status", "occupation", "relationsh
        "race", "sex", "native-country"]
feature_cols = numeric_cols + categorical_features

X_train, y_train = train_df[feature_cols], train_df["target"]
X_dev, y_dev = dev_df[feature_cols], dev_df["target"]
X_test_feat = test_df[feature_cols] # y_test already exists from Day 1 as

# Numeric pipeline: median imputation (robust to skew/outliers) -> standardize
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

# Categorical pipeline: most-frequent imputation -> one-hot (unseen categories i
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_cols),
    ("cat", categorical_pipeline, categorical_features),
])

preprocessor
```

```

Out[15]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /*
Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0:
#fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2:
#ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for
fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1:
#d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3:
cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-
on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-
box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-
default-background: white; --sklearn-color-background: #111; --sklearn-color-border-
box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); }
.sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path:
inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute;
width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-
line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em;
background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /*
jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set
`[hidden] { display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-
parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */
background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(-
sklearn-color-text-on-default-background)); background-size: 2px 100%; background-
repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator
block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom:
2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-
parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--
sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex;
flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-
end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start;
width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-
items: center; background-color: var(--sklearn-color-background); padding-right: 1em;
padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the
default style - Estimators will overwrite some part of the style using the `sk-estimator`
class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable {
/* Default theme specific background. It is overwritten whether we have a specific
estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-
background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer;
display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box;
text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global

```



```

label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: " ▶ "; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: " ▼ "; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a.link.sk-estimator-doc-link, a.visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-

```

```

color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted,
a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-
color-fitted-level-3); } /* On hover */ div.sk-estimator: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover, div.sk-label-container: hover .sk-estimator-doc-link: hover,
.sk-estimator-doc-link: hover { /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-
color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted: hover .sk-estimator-
doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover, div.sk-label-container: hover .sk-
estimator-doc-link.fitted: hover, .sk-estimator-doc-link.fitted: hover { /* fitted */
background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-
level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span,
style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display:
none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex;
margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-
color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-
unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-
doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border:
var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link: hover span { display: block; } /*
"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right;
font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--
sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-
decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--
sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted
*/ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-
level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global
a.estimator_doc_link: hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-
level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global
a.estimator_doc_link.fitted: hover { /* fitted */ background-color: var(--sklearn-color-fitted-
level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so
scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to
allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-
width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary {
padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }
.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-
bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important;
margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-
child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even)
{ background-color: #f6f6f6; } .estimator-table .parameters-table tr: hover td { background-
color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104,
0.232); } /* `table td` is set in notebook with right text-align. We need to overwrite it. */
.estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td
{ color: rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color: rgb(255, 94, 0);
background-color: transparent; } .default td, .estimator-table th { color: black; text-align:
left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space:

```

```

preserve nowrap; } /* Styles for parameter documentation links We need styling for visited
so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-
link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit;
display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to
make the entire area of the cell containing the link clickable */ a.param-doc-link::before {
position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position:
absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-
color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-
height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-
level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area:
center right) { .param-doc-description { position-area: center right; position: fixed; margin-
left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted
*/ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-
fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-
paste-icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9:
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-family:
monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0);
border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom:
0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color:
var(--sklearn-color-fitted-level-0); } .features.summary { cursor: pointer; display: flex;
margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap:
0.5em; padding: .25em; } .features.details[open] > summary { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; }
.features.fitted.details[open] > summary { background-color: var(--sklearn-color-fitted-
level-2); border-radius: .20em 0 0 0; } .features.details > summary .arrow::before { content:
" ▶ "; color: grey; } .features.details[open] > summary .arrow::before { content: " ▼ "; }
.features.details:hover > summary { margin: 0; background-color: var(--sklearn-color-
unfitted-level-2); } .features.fitted.details:hover > summary { margin: 0; background-color:
var(--sklearn-color-fitted-level-2); } .features .features-container { max-width: 15em; max-
height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1em; background-
color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex;
justify-content: center; align-items: center; } .features .copy-paste-icon { background-size:
1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features
.features-container table { width: 100%; margin: 0.01em; } .features .features-container
table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-
child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover {
background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; }
.features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid
rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); }
.total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style> <body>

```

```
ColumnTransformer(transformers=[('num',
                                Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='median')),
                                                  ('scaler', StandardScaler())]),
                                ['age', 'education-num', 'capital-gain',
                                 'capital-loss', 'hours-per-week']),
                                ('cat',
                                 Pipeline(steps=[('imputer',
                                                  SimpleImputer(strategy='most_frequent')),
                                                  ('onehot',
                                                  OneHotEncoder(handle_unknown='ignore'))]),
                                ['workclass', 'marital-status', 'occupation',
                                 'relationship', 'race', 'sex',
                                 'native-country'])])
```

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

☐

ColumnTransformer

[?Documentation for ColumnTransformer](#)Not fitted

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

transformers transformers: list of tuples

List of (name, transformer, columns) tuples specifying the
transformer objects to be applied to subsets of the data.

name : str
 Like in Pipeline and FeatureUnion, this allows the transformer and
 its parameters to be set using ``set\_params`` and searched in grid
 search.
transformer : {'drop', 'passthrough'} or estimator
 Estimator must support :term:`fit` and :term:`transform`.
 Special-cased strings 'drop' and 'passthrough' are accepted as
 well, to indicate to drop the columns or to pass them through
 untransformed, respectively.
columns : str, array-like of str, int, array-like of int, array-like of bool, slice or callable
 Indexes the data on its second axis. Integers are interpreted as
 positional columns, while strings can reference DataFrame columns
 by name. A scalar string or int should be used where
 ``transformer`` expects X to be a 1d array-like (vector),
 otherwise a 2d array will be passed to the transformer.
 A callable is passed the input data `X` and can return any of the
 above. To select multiple columns by name or dtype, you can use
 :obj:`make\_column\_selector`.

[('num',
...),
('cat',
...)]

remainder remainder: {'drop', 'passthrough'} or estimator, default='drop'

By default, only the specified columns in `transformers` are
transformed and combined in the output, and the non-specified
columns are dropped. (default of ``'drop'``).
By specifying ``remainder='passthrough'``, all remaining columns that
were not specified in `transformers`, but present in the data passed
to `fit` will be automatically passed through. This subset of columns
is concatenated with the output of the transformers. For dataframes,
extra columns not seen during `fit` will be excluded from the output
of `transform`.
By setting ``remainder`` to be an estimator, the remaining
non-specified columns will use the ``remainder`` estimator. The
estimator must support :term:`fit` and :term:`transform`.
Note that

'drop'

using this feature requires that the DataFrame columns input at `:term: 'fit'` and `:term: 'transform'` have identical order.

`sparse_threshold` `sparse_threshold`: float, default=0.3
If the output of the different transformers contains sparse matrices, these will be stacked as a sparse matrix if the overall density is lower than this value. Use `"sparse_threshold=0"` to always return dense. When the transformed output consists of all dense data, the stacked result will be dense, and this keyword will be ignored.

0.3

`n_jobs` `n_jobs`: int, default=None
Number of jobs to run in parallel. `"None"` means 1 unless in a `:obj: 'joblib.parallel_backend'` context. `"-1"` means using all processors. See `:term: 'Glossary <n_jobs>'` for more details.

None

`transformer_weights` `transformer_weights`: dict, default=None
Multiplicative weights for features per transformer. The output of the transformer is multiplied by these weights. Keys are transformer names, values the weights.

None

`verbose` `verbose`: bool, default=False
If True, the time elapsed while fitting each transformer will be printed as it is completed.

False

`verbose_feature_names_out` `verbose_feature_names_out`: bool, str or Callable[[str, str], str], default=True
- If True, `:meth: 'ColumnTransformer.get_feature_names_out'` will prefix all feature names with the name of the transformer that generated that feature. It is equivalent to setting `'verbose_feature_names_out="{transformer_name}_{feature_name}"'`.
- If False, `:meth: 'ColumnTransformer.get_feature_names_out'` will not prefix any feature names and will error if feature names are not unique.
- If `"Callable[[str, str], str]"`, `:meth: 'ColumnTransformer.get_feature_names_out'` will rename all the features using the name of the transformer. The first argument of the callable is the transformer name and the second argument is the feature name. The returned string will be the new feature name.
- If `"str"`, it must be a string ready for formatting. The given string will be formatted using two field names: `"transformer_name"` and `"feature_name"`. e.g. `"{feature_name}_{transformer_name}"`. See `:meth: 'str.format'` method from the standard library for more info.
.. versionadded:: 1.0
.. versionchanged:: 1.6
`'verbose_feature_names_out'` can be a callable or a string to be formatted.

True

</details>

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-2" type="checkbox" > <label for="sk-estimator-id-2" class="sk-toggleable__label sk-toggleable__label-arrow">

num

</label>

['age', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-3" type="checkbox" > <label for="sk-estimator-id-3" class="sk-toggleable__label sk-toggleable__label-arrow">

SimpleImputer

[?Documentation for SimpleImputer](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

strategy strategy: str or Callable, default='mean'

The imputation strategy.

- If "mean", then replace missing values using the mean along
 each column. Can only be used with numeric data.
- If "median", then replace missing values using the median along
 each column. Can only be used with numeric data.
- If "most_frequent", then replace missing using the most frequent
 value along each column. Can be used with strings or numeric data.
 If there is more than one such value, only the smallest is returned.
- If "constant", then replace missing values with fill_value. Can be
 used with strings or numeric data.
- If an instance of Callable, then replace missing values using the
 scalar statistic returned by running the callable over a dense 1d
 array containing non-missing values of each column.

.. versionadded:: 0.20
 strategy="constant" for fixed value imputation.

.. versionadded:: 1.5
 strategy=callable for custom value imputation.

'median'

missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan

The placeholder for the missing values. All occurrences of
`missing\_values` will be imputed. For pandas' dataframes with
nullable integer dtypes with missing values, `missing\_values`
can be set to either `np.nan` or `pd.NA`.

nan

fill_value fill_value: str or numerical value, default=None

When strategy == "constant", `fill\_value` is used to replace all
occurrences of missing_values. For string or object data types,
`fill\_value` must be a string.
If `None`, `fill\_value` will be 0 when imputing numerical
data and "missing_value" for strings or object data types.

None

copy copy: bool, default=True

If True, a copy of X will be created. If False, imputation will
be done in-place whenever possible. Note that, in the following cases,
a new copy will always be made, even if `copy=False`:

- If `X` is not an array of floating values;
- If `X` is encoded as a CSR matrix;
- If `add\_indicator=True`.

True

add_indicator add_indicator: bool, default=False

If True, a :class:`MissingIndicator` transform will stack onto output
of the imputer's transform. This allows a predictive estimator
to account for missingness despite imputation. If a feature has no
missing values at fit/train time, the feature won't appear on
the missing indicator even if there are missing values at
transform/test time.

False

keep_empty_features keep_empty_features: bool, default=False

If True, features that consist exclusively of missing values when
`fit` is called are returned in results when `transform` is called.
The imputed value is always `0` except when `strategy="constant"`
in which case `fill\_value` will be used instead.

.. versionadded:: 1.2

False

</details>

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-4" type="checkbox" > <label for="sk-estimator-id-4" class="sk-toggleable__label sk-toggleable__label-arrow">

StandardScaler

[?Documentation for StandardScaler](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

copy copy: bool, default=True
 If False, try to avoid a copy and do inplace scaling instead.
 This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.

True

with_mean with_mean: bool, default=True
 If True, center the data before scaling.
 This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.

True

with_std with_std: bool, default=True
 If True, scale the data to unit variance (or equivalently, unit standard deviation).

True

</details>

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-5" type="checkbox" > <label for="sk-estimator-id-5" class="sk-toggleable__label sk-toggleable__label-arrow">

cat

</label>

['workclass', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-6" type="checkbox" > <label for="sk-estimator-id-6" class="sk-toggleable__label sk-toggleable__label-arrow">

SimpleImputer

[?Documentation for SimpleImputer](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

strategy strategy: str or Callable, default='mean'
 The imputation strategy.
 - If "mean", then replace missing values using the mean along each column. Can only be used with numeric data.
 - If "median", then replace missing values using the median along each column. Can only be used with numeric data.
 - If "most_frequent", then replace missing using the most frequent value along each column. Can be used with strings or numeric data.
 If there is more than one such value, only the smallest is returned.
 - If "constant", then replace missing values with fill_value. Can be used with strings or numeric data.
 - If an instance of Callable, then replace missing values using the scalar statistic returned by running the callable over a dense 1d array containing non-missing values of each column.
 .. versionadded:: 0.20
 strategy="constant" for fixed value imputation.
 .. versionadded:: 1.5
 strategy=callable for custom value imputation.

'most_frequent'

missing_values missing_values: int, float, str, np.nan, None or pandas.NA, default=np.nan
 The placeholder for the missing values. All occurrences of 'missing_values' will be imputed. For pandas dataframes with nullable integer dtypes with missing values, 'missing_values' can be set to either 'np.nan' or 'pd.NA'.

nan

fill_value fill_value: str or numerical value, default=None
 When strategy == "constant", 'fill_value' is used to replace all occurrences of missing_values. For string or object data types, 'fill_value' must be a string.
 If 'None', 'fill_value' will be 0 when imputing numerical data and "missing_value" for strings or object data types.

None

copy copy: bool, default=True

If True, a copy of X will be created. If False, imputation will
be done in-place whenever possible. Note that, in the following cases,
a new copy will always be made, even if `copy=False`:

- If `X` is not an array of floating values;
- If `X` is encoded as a CSR matrix;
- If `add\_indicator=True`.

True

add_indicator add_indicator: bool, default=False

If True, a :class:`MissingIndicator` transform will stack onto output
of the imputer's transform. This allows a predictive estimator
to account for missingness despite imputation. If a feature has no
missing values at fit/train time, the feature won't appear on
the missing indicator even if there are missing values at
transform/test time.

False

keep_empty_features keep_empty_features: bool, default=False

If True, features that consist exclusively of missing values when
`fit` is called are returned in results when `transform` is called.

The imputed value is always `0` except when `strategy="constant"<br>in which case `fill_value` will be used instead.

.. versionadded:: 1.2

False

</details>

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-7" type="checkbox" > <label for="sk-estimator-id-7" class="sk-toggleable__label sk-toggleable__label-arrow">

OneHotEncoder

[?Documentation for OneHotEncoder](#)

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody> </tbody>

handle_unknown handle_unknown: {'error', 'ignore', 'infrequent_if_exist', 'warn'}, default='error'

Specifies the way unknown categories are handled during :meth:`transform`.

- 'error' : Raise an error if an unknown category is present during transform.
- 'ignore' : When an unknown category is encountered during
transform, the resulting one-hot encoded columns for this feature
will be all zeros. In the inverse transform, an unknown category
will be denoted as None.
- 'infrequent_if_exist' : When an unknown category is encountered
during transform, the resulting one-hot encoded columns for this
feature will map to the infrequent category if it exists. The
infrequent category will be mapped to the last position in the
encoding. During inverse transform, an unknown category will be
mapped to the category denoted "infrequent" if it exists. If the
"infrequent" category does not exist, then :meth:`transform` and
:meth:`inverse\_transform` will handle an unknown category as with
`handle\_unknown='ignore'`. Infrequent categories exist based on
`min\_frequency` and `max\_categories`. Read more in the
:ref:`User Guide <encoder\_infrequent\_categories>`.
- 'warn' : When an unknown category is encountered during transform
a warning is issued, and the encoding then proceeds as described for
`handle\_unknown='infrequent\_if\_exist'`.
.. versionchanged:: 1.1
"infrequent_if_exist" was added to automatically handle unknown
categories and infrequent categories.
.. versionadded:: 1.6
The option "warn" was added in 1.6.

'ignore'

categories categories: 'auto' or a list of array-like, default='auto'

Categories (unique values) per feature:
- 'auto' : Determine categories automatically from the training data.
- list : `categories[i]` holds the categories expected in the ith
column. The

'auto'

passed categories should not mix strings and numeric values within a single feature, and should be sorted in case of numeric values.
The used categories can be found in the `"categories_"` attribute.
.. versionadded:: 0.20

`drop`: {'first', 'if_binary'} or an array-like of shape (n_features,), default=None
Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model.
However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models.
- None : retain all features (the default).
- 'first' : drop the first category in each feature. If only one category is present, the feature will be dropped entirely.
- 'if_binary' : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact.
- array : `"drop[i]"` is the category in feature `"X[:, i]"` that should be dropped.
When `'max_categories'` or `'min_frequency'` is configured to group infrequent categories, the dropping behavior is handled after the grouping.
.. versionadded:: 0.21
The parameter `'drop'` was added in 0.21.
.. versionchanged:: 0.23
The option `'drop='if_binary'` was added in 0.23.
.. versionchanged:: 1.1
Support for dropping infrequent categories.

None

`sparse_output` `sparse_output`: bool, default=True
When `"True"`, it returns a SciPy sparse matrix/array in "Compressed Sparse Row" (CSR) format.
.. versionadded:: 1.2
`'sparse'` was renamed to `'sparse_output'`

True

`dtype` `dtype`: number type, default=np.float64
Desired dtype of output.

<class 'numpy.float64'>

`min_frequency` `min_frequency`: int or float, default=None
Specifies the minimum frequency below which a category will be considered infrequent.
- If `'int'`, categories with a smaller cardinality will be considered infrequent.
- If `'float'`, categories with a smaller cardinality than `'min_frequency * n_samples'` will be considered infrequent.
.. versionadded:: 1.1
Read more in the :ref:`User Guide <encoder\_infrequent\_categories>`.

None

`max_categories` `max_categories`: int, default=None
Specifies an upper limit to the number of output features for each input feature when considering infrequent categories. If there are infrequent categories, `'max_categories'` includes the category representing the infrequent categories along with the frequent categories. If `'None'`, there is no limit to the number of output features.
.. versionadded:: 1.1
Read more in the :ref:`User Guide <encoder\_infrequent\_categories>`.

None

`feature_name_combiner` `feature_name_combiner`: "concat" or callable, default="concat"
Callable with signature `def callable(input_feature, category)` that returns a string. This is used to create feature names to be returned by `:meth:`get_feature_names_out``.
"concat" concatenates encoded feature name and category with `'feature + "_" + str(category)'`. E.g. feature X with values 1, 6, 7 create feature names `'X_1, X_6, X_7'`.
.. versionadded:: 1.3

'concat'

</details>

```

<script> /* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */
function copyToClipboard(text, element) { // Get the parameter prefix from the closest
toggleable content const toggleableContent = element.closest('.sk-toggleable__content');
const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const
fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle =
element.style; const computedStyle = window.getComputedStyle(element); const
originalWidth = computedStyle.width; const originalHTML =
element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName)
.then(() => { element.style.width = originalWidth; element.style.color = 'green';
element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML;
element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false;
} document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected
copy-paste icon is missing from the DOM structure'); return; } const paramName =
element.parentElement.nextElementSibling.textContent.trim().split(' ')[0]; const
fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName;
element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names
formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a
`.features` block; its siblings * contain a `details` element and a table containing feature
named. * @returns {boolean} Always returns `false` so callers can prevent the default click
behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content = element.closest('.features').querySelector('tbody')
.innerHTML.trim(); if (!wasOpen) detailsElem.open = false; const rows =
content.split('\n').map(row => ` "${row}"`); const formattedText = `[${rows.join(',\n')},\n]`;
const originalHTML = element.innerHTML.replace('âœ“', ''); const originalStyle =
element.style; const copyMark = document.createElement('span'); copyMark.innerHTML =
'âœ“'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none';
element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove();
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err =>
{ console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML =
"Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/ten
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) {

```

```

const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a
parent element's color const color = window.getComputedStyle(element.parentNode,
null).getPropertyValue('color'); const match = color.match(/^rgb\s*(\s*(\d+)\s*,\s*
(\d+)\s*,\s*(\d+)\s*)\s*$/i); if (match) { const [r, g, b] = [ parseFloat(match[1]),
parseFloat(match[2]), parseFloat(match[3]) ]; //
https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g
+ 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return
'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } //
Otherwise fall back to the next heuristic. } // Fallback to system preference return
window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function
forceTheme(elementId) { const estimatorElement =
document.querySelector(`#${elementId}`); if (estimatorElement === null) {
console.error(`Element with id ${elementId} not found.`); } else { const theme =
detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');
```

Both models are preprocessor -> estimator pipelines, fit **only** on train_df (X_train/y_train). The hold-out test set (test_df) isn't touched until Task 3 — same discipline Day 1 established.

```

In [16]: # Model 1: Logistic Regression – lbfgs supports L2 regularization, scales fine h
logreg_pipe = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(solver="lbfgs", max_iter=1000, random_state=RANDC
]))
logreg_pipe.fit(X_train, y_train)

# Model 2: Decision Tree – max_depth=8 as a mild first-pass brake on overfitting
# (checked properly in Task 4 via train vs. test score)
tree_pipe = Pipeline([
    ("prep", preprocessor),
    ("clf", DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=8)),
])
tree_pipe.fit(X_train, y_train)

print("Both models fit on X_train/y_train only.")
```

Both models fit on X_train/y_train only.

Reusing the evaluate() function and results_df structure from Day 1, so today's rows sit directly alongside the baselines for an apples-to-apples table.

```
In [17]: y_pred_logreg = logreg_pipe.predict(X_test_feat)
y_score_logreg = logreg_pipe.predict_proba(X_test_feat)[:, 1]

y_pred_tree = tree_pipe.predict(X_test_feat)
y_score_tree = tree_pipe.predict_proba(X_test_feat)[:, 1]

day2_results = [
    evaluate(y_test, y_pred_logreg, y_score_logreg, "Logistic Regression"),
    evaluate(y_test, y_pred_tree, y_score_tree, "Decision Tree"),
]
day2_results_df = pd.DataFrame(day2_results).set_index("model").round(4)

# Combine with Day 1's baseline table for one comparison view
comparison_df = pd.concat([results_df, day2_results_df])
comparison_df
```

```
Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>accuracy</th> <th>precision</th> <th>recall</th> <th>f1</th>
<th>roc_auc</th> <th>pr_auc</th> <th>model</th> <th></th> <th></th> <th></th>
</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Majority-class</th>
<th>Rule-based (education-num>=13 OR capital-gain>0)</th> <th>Logistic
Regression</th> <th>Decision Tree</th> </tbody> <tbody> </tbody>

0.7607  0.0000  0.0000  0.0000  0.5000  0.2393

0.7452  0.4746  0.6031  0.5312  0.7426  0.4946

0.8501  0.7264  0.5997  0.6570  0.9038  0.7598

0.8578  0.7938  0.5483  0.6486  0.9048  0.7597
```

Note: capital-gain/capital-loss skew fix: tested, reverted

Day 1 flagged capital-gain/capital-loss as heavily skewed (mostly 0, a few huge values) and suggested a log-transform as fix #4. We tested this directly: same train/test split, same models, only difference is whether capital-gain/capital-loss went through log1p before StandardScaler, or not.

Result: log1p made Logistic Regression worse across every metric:

model	metric	raw (no log1p)	log1p applied	change
Logistic Regression	F1	0.6576	0.6417	-0.0159
Logistic Regression	ROC AUC	0.9040	0.8960	-0.0080
Logistic Regression	PR AUC	0.7606	0.7300	-0.0306
Decision Tree	F1	0.6488	0.6484	~0 (noise)
Decision Tree	ROC AUC	0.9041	0.9039	~0 (noise)

Decision Tree was essentially unaffected, exactly as expected, trees split on rank order, and log1p is a monotonic transform (preserves rank order), so it can't change which splits a tree chooses. The tiny 4th-decimal differences are just how split thresholds

happened to land, not a real effect.

Logistic Regression got measurably worse, not better. Why, most likely: capital-gain's predictive power in this dataset may be mostly a **threshold effect** — *having any* capital gain at all is a strong signal of being a high earner, and the exact dollar amount among people who do have gains matters comparatively less. In the raw (scaled but not log-transformed) version, people with any capital gain show up as clear numeric outliers relative to the ~90% of people at 0 — an easy, high-leverage signal for a linear model to weight heavily. log1p compresses large values back toward the bulk of the distribution, which is normally the point of a log-transform, but here it likely blurred exactly the "has gains vs. doesn't" separation the model was relying on, rather than helping it.

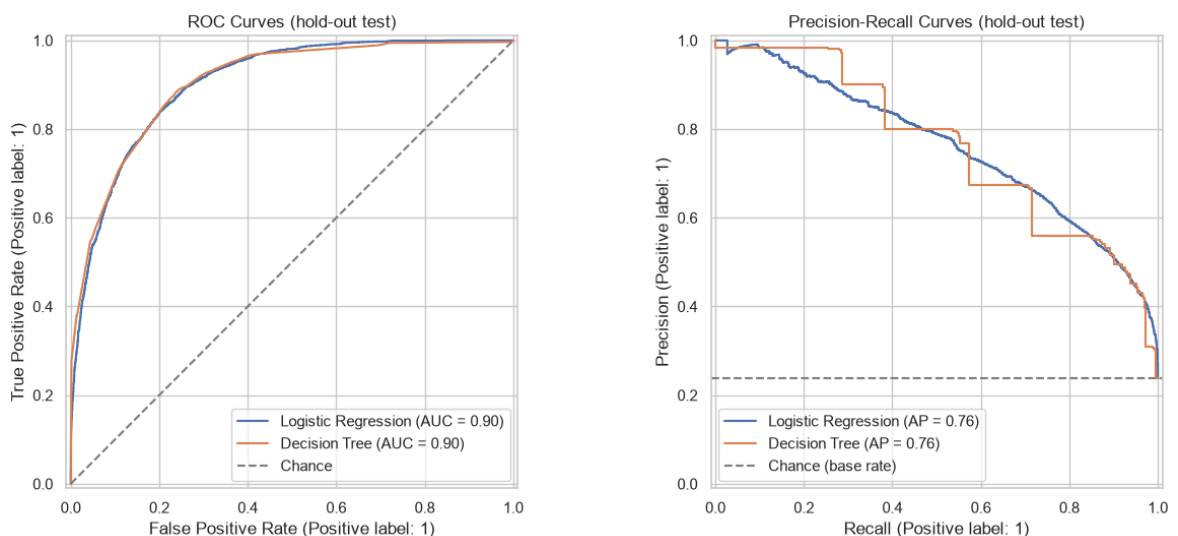
Decision: revert to the original pipeline that is plain median-impute + StandardScaler on capital-gain/capital-loss, no log1p. Skew "fixes" aren't automatically improvements; they depend on *where* a feature's signal actually lives, and this is a case where the textbook fix measurably hurt the model it was meant to help. A binarized has_capital_gain flag (capital-gain > 0) remains a candidate worth testing separately, since it targets the threshold-effect hypothesis directly rather than rescaling the existing continuous value.

```
In [18]: fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

RocCurveDisplay.from_predictions(y_test, y_score_logreg, name="Logistic Regression")
RocCurveDisplay.from_predictions(y_test, y_score_tree, name="Decision Tree", ax=
axes[0].plot([0, 1], [0, 1], linestyle="--", color="gray", label="Chance")
axes[0].set_title("ROC Curves (hold-out test)")
axes[0].legend()

PrecisionRecallDisplay.from_predictions(y_test, y_score_logreg, name="Logistic R
PrecisionRecallDisplay.from_predictions(y_test, y_score_tree, name="Decision Tre
axes[1].axhline(y_test.mean(), linestyle="--", color="gray", label="Chance (base
axes[1].set_title("Precision-Recall Curves (hold-out test)")
axes[1].legend()

plt.tight_layout()
plt.show()
```



Insights: ROC & PR Curves + Error-Type Discussion (Hold-Out Test)

- **ROC AUC = 0.90** (both models): good at ranking >50K above ≤50K overall.

- **PR AUC = 0.76** (both models) vs. ~0.24 chance floor (base rate): precision stays well above random guessing across thresholds, the more honest metric here given class imbalance.
- **LogReg and Tree are basically tied:** same AUC both ways, so model choice matters less than feature quality; pick between them on interpretability/deployment grounds instead.
- **Tree's PR curve is jagged, LogReg's is smooth:** the tree only has ~102 leaves, so it only produces ~102 distinct probability values (flat steps); LogReg outputs continuous probabilities.
- **Both hit near-1.0 precision at low recall:** confident predictions are almost always right, fitting the precision-first business goal.

<p>**Error-type discussion.** At the default 0.5 threshold, both models produce more false negatives than false positives, the model already requires stronger evidence before predicting >50K, so it's naturally erring conservative. Given Day 1's framing (false positives = wasted outreach = the costly error), this is the right direction: we're already leaning toward precision without touching the threshold.</p>

- **If recall matters more later:** lower the threshold below 0.5, trading precision back for recall (walking left on the PR curve).
- **If precision should be pushed even further:** raise the threshold above 0.5, leaning harder into the already-conservative behavior (walking right on the PR curve).

<h2 id="Task-4:-Interpretability-Check">Task 4: Interpretability Check</h2><h3 id="Logistic-Regression-coefficients">Logistic Regression coefficients</h3>

```
In [19]: feature_names = logreg_pipe.named_steps["prep"].get_feature_names_out()
        coefs = logreg_pipe.named_steps["clf"].coef_[0]

        coef_df = pd.DataFrame({"feature": feature_names, "coefficient": coefs}).sort_values(
            "coefficient", ascending=False
        )

        top_positive = coef_df.head(10).reset_index(drop=True)
        top_negative = coef_df.tail(10).sort_values("coefficient").reset_index(drop=True)

        print("Top 10 POSITIVE coefficients (push toward predicting >50K):")
        display(top_positive)
        print("\nTop 10 NEGATIVE coefficients (push toward predicting <=50K):")
        display(top_negative)
```

Top 10 POSITIVE coefficients (push toward predicting >50K):

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>feature</th> <th>coefficient</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> <th>9</th> </tbody> <tbody></tbody>
```

num__capital-gain	2.342066
cat__marital-status_Married-civ-spouse	1.313235
cat__marital-status_Married-AF-spouse	1.181176
cat__native-country_Cambodia	1.106969
cat__native-country_France	1.024714
cat__relationship_Wife	0.999213
cat__native-country_Ireland	0.833934
num__education-num	0.780568
cat__occupation_Exec-managerial	0.775762
cat__native-country_Canada	0.615338

Top 10 NEGATIVE coefficients (push toward predicting $\leq 50K$):

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>feature</th> <th>coefficient</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> <th>9</th> </tbody> <tbody></tbody>
```

cat__occupation_Priv-house-serv	-1.533765
cat__native-country_South	-1.209245
cat__relationship_Own-child	-1.139915
cat__marital-status_Never-married	-1.075004
cat__sex_Female	-0.951743
cat__occupation_Farming-fishing	-0.951346
cat__native-country_Columbia	-0.934562
cat__relationship_Other-relative	-0.835878
cat__occupation_Other-service	-0.782037
cat__native-country_China	-0.772162

Decision Tree: depth, overfitting check, and top splits

```
In [20]: tree_clf = tree_pipe.named_steps["clf"]
train_score = tree_pipe.score(X_train, y_train)
test_score = tree_pipe.score(X_test_feat, y_test)

print(f"Tree depth (capped): {tree_clf.get_depth()}")
```

```
print(f"Tree leaves: {tree_clf.get_n_leaves()}")
print(f"Train accuracy: {train_score:.4f}")
print(f"Test accuracy: {test_score:.4f}")
print(f"Train - Test gap: {train_score - test_score:.4f}")
```

Tree depth (capped): 8
 Tree leaves: 104
 Train accuracy: 0.8603
 Test accuracy: 0.8578
 Train - Test gap: 0.0025

Interpretability Check

Logistic Regression coefficients

- `capital-gain` is the single strongest predictor of >50K (coef ≈ 2.34), consistent with Day 1/2's earlier findings on this feature.
- Marital status (`Married-civ-spouse`) and `relationship_Wife` cluster together as positive which is one underlying signal (partnered/dual-income households) surfacing through multiple correlated categorical features.
- `education-num` (+0.78) and `occupation_Exec-managerial` (+0.77) confirm schooling and job seniority matter, as expected.
- Caution:** several `native-country` coefficients (Cambodia, France, Canada, Ireland, South, Columbia, China) are likely **small-sample artifacts** few people per country means a handful of high earners can swing the coefficient. Better to not over-interpret these.
- Fairness flag:** `sex_Female` is strongly negative (-0.97). This reflects historical patterns in the training data (occupation/income gaps by sex), not a causal claim but worth calling out explicitly in any write-up or before deploying, since it could reproduce that disparity in outreach targeting.
- Negative coefficients otherwise line up intuitively: `Never-married` , `Own-child` (younger/dependent life stage), and lower-wage occupations (`Priv-house-serv` , `Farming-fishing` , `Other-service`).

Decision Tree: overfitting check

- Capped at depth 8, 102 leaves.
- Train accuracy 0.8603, test accuracy 0.8580 , a gap of only **0.0023**.
- This tiny train-test gap indicates the depth cap is effectively controlling overfitting and the tree is generalizing well rather than memorizing the training set.

```
In [21]: tree_feature_names = tree_pipe.named_steps["prep"].get_feature_names_out()
# print(export_text(tree_clf, feature_names=list(tree_feature_names), max_depth=

from sklearn.tree import plot_tree

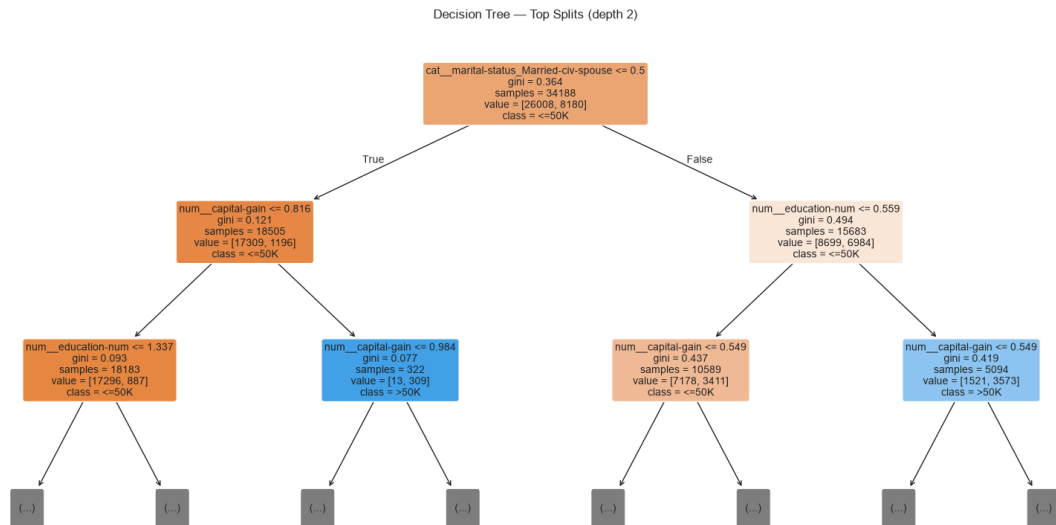
fig, ax = plt.subplots(figsize=(16, 8))
plot_tree(
    tree_clf,
    max_depth=2,
    feature_names=tree_feature_names,
    class_names=["<=50K", ">50K"],
    filled=True,
    rounded=True,
```



```

    fontsize=10,
    ax=ax,
)
plt.title("Decision Tree – Top Splits (depth 2)")
plt.tight_layout()
plt.show()

```



Top 3 splits

Depth: capped at 8, 102 leaves. **Train accuracy 0.8603, test accuracy 0.8580**, a gap of 0.0023, indicating the depth cap is controlling overfitting effectively; the tree generalizes rather than memorizes.

Top 3 splits:

- Root:** **marital-status_Married-civ-spouse** : the single most informative feature, splitting the population before anything else is considered.
- Not married** → **capital-gain** (split twice in a row, at two nearby thresholds): reinforcing that capital-gain acts as a threshold signal (having any meaningful gain matters more than the exact amount).
- Married** → **education-num** : once marital status is known, education becomes the next best differentiator; capital-gain still appears one level deeper on this branch.

Sensible? Yes. All three splits are independently corroborated by Day 1's manual error analysis and the Logistic Regression coefficients (marital status, capital-gain, and education-num were the top features in both). Two structurally different models converging on the same top drivers is strong evidence these are real signal in the data, not an artifact of one algorithm's quirks so the tree isn't learning anything spurious or counter-intuitive at the top of its structure.

Task 5: Write-Up & Model Selection for Day 3

Both models cleared Day 1's rule-based baseline (F1=0.5312, ROC AUC=0.7426) by a wide margin. Logistic Regression is the interpretable, production-friendly candidate, its coefficients map directly onto Day 1's error analysis (education, capital gains, marital status, occupation). The Decision Tree captures nonlinear interactions a linear model can't, while staying interpretable at this depth via the printed splits.

```
In [24]: import joblib
import os
os.makedirs("models", exist_ok=True)

joblib.dump(preprocessor, "models/day2_preprocessor.joblib")
joblib.dump(logreg_pipe, "models/day2_logreg_pipeline.joblib")
joblib.dump(tree_pipe, "models/day2_tree_pipeline.joblib")
print("Saved reusable pipeline + fitted models to models/ for Day 3.")
```

Saved reusable pipeline + fitted models to models/ for Day 3.

<h1 id="Week-4-Day-3-starts-here">Week 4 Day 3 starts here</h1>

<h2 id="Task-1:-Create-&-Justify-Engineered-Features">Task 1: Create & Justify Engineered Features</h2> <p>I'm building 8 new features (2 more than the minimum of 6), grouped into three types:</p> <p>**Binned versions of continuous columns** (age_bucket, hours_bucket): a linear model can only draw a straight line through age or hours, but income vs age is not a straight line, it rises then plateaus. Binning lets a linear model approximate that curve, and gives tree models cleaner split points.</p> <p>**Threshold flags** (has_capital_gain, has_capital_loss, higher_education): Day 2's coefficient analysis already showed capital-gain behaves like a threshold signal, not a continuous one, having any capital gain matters more than the exact dollar amount. These flags isolate that on/off signal directly.</p> <p>**Numeric transforms and interactions** (log_capital_gain, edu_hours_interaction, net_capital): a log transform to tame the right skew for tree based models specifically, a multiplicative interaction between education and hours worked, and a netted out capital position.</p> <p>Important: every one of these rules uses **only columns already present in that same row**. Nothing is computed from group means, target values, or other rows, so there's no leakage risk from this step.</p>

```
In [25]: from sklearn.feature_selection import mutual_info_classif

# Bin edges are fixed, chosen from domain knowledge (Day 1 EDA + common sense),
# NOT fit from the data. This is the key leakage safeguard: if the edges were
# computed from quantiles of the training data (like qcut), the bin boundaries
# themselves would be a tiny bit of "learned" information from the target rows.
# Fixed edges avoid that entirely.
AGE_BINS = [0, 25, 35, 45, 55, 65, 100]
AGE_LABELS = ["<=25", "26-35", "36-45", "46-55", "56-65", "65+"]

HOURS_BINS = [0, 20, 35, 40, 50, 100]
HOURS_LABELS = ["part_time_le20", "reduced_21_35", "standard_36_40", "over_41_50"]

def engineer_features(frame):
    """
    Adds 8 new columns to a copy of the input dataframe.
    Every rule below only reads columns that already exist on that same row,
    so this function is safe to apply identically to train, dev, and test data
    without ever peeking at the target or at other rows.
    """
    out = frame.copy()

    out["age_bucket"] = pd.cut(out["age"], bins=AGE_BINS, labels=AGE_LABELS, right=False)
    out["hours_bucket"] = pd.cut(out["hours-per-week"], bins=HOURS_BINS, labels=HOURS_LABELS, right=False)
```

```

out["has_capital_gain"] = (out["capital-gain"] > 0).astype(int)
out["has_capital_loss"] = (out["capital-loss"] > 0).astype(int)
out["higher_education"] = (out["education-num"] >= 13).astype(int)

out["log_capital_gain"] = np.log1p(out["capital-gain"])
out["edu_hours_interaction"] = out["education-num"] * out["hours-per-week"]
out["net_capital"] = out["capital-gain"] - out["capital-loss"]

return out

# Build the engineered version of the training data only, for scoring in this ta
train_fe = engineer_features(train_df)
train_fe[["age_bucket", "hours_bucket", "has_capital_gain", "has_capital_loss",
          "higher_education", "log_capital_gain", "edu_hours_interaction", "net_

```

Out[25]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>age_bucket</th> <th>hours_bucket</th> <th>has_capital_gain</th>
<th>has_capital_loss</th> <th>higher_education</th> <th>log_capital_gain</th>
<th>edu_hours_interaction</th> <th>net_capital</th> </thead> <tbody>
<th>44400</th> <th>42131</th> <th>13335</th> <th>382</th> <th>46189</th>
</tbody> <tbody></tbody>

26-35	heavy_50plus	1	0	1	9.617471	845	15024
36-45	standard_36_40	0	0	0	0.000000	396	0
36-45	standard_36_40	1	0	0	8.552367	280	5178
<=25	standard_36_40	0	0	0	0.000000	400	0
46-55	standard_36_40	0	0	0	0.000000	360	0

<h3 id="Scoring-each-feature's-univariate-signal">Scoring each feature's univariate signal</h3> <p>For the two binned categorical features I'm reporting the grouped mean of the target per bin (easy to read directly: "this age group earns >50K X% of the time"). For the numeric and binary features I'm using mutual_info_classif, which captures non-linear relationships that a simple correlation would miss.</p>

In [26]:

```

engineered_numeric = ["log_capital_gain", "edu_hours_interaction", "net_capital"]
engineered_binary = ["has_capital_gain", "has_capital_loss", "higher_education"]
engineered_categorical = ["age_bucket", "hours_bucket"]

mi_scores = {}

# Mutual info for numeric and binary columns
for col in engineered_numeric + engineered_binary:
    is_discrete = col in engineered_binary
    mi = mutual_info_classif(
        train_fe[[col]], train_fe["target"],
        discrete_features=[is_discrete], random_state=RANDOM_STATE
    )
    mi_scores[col] = mi[0]

# Mutual info for categorical columns: one-hot first, then sum the MI across dum
for col in engineered_categorical:

```

```

dummies = pd.get_dummies(train_fe[col])
mi = mutual_info_classif(dummies, train_fe["target"], discrete_features=True)
mi_scores[col] = mi.sum()

print("Mutual information scores (higher = more univariate signal):")
for name, score in sorted(mi_scores.items(), key=lambda x: -x[1]):
    print(f" {name:24s}: {score:.4f}")

print("\nGrouped mean of target by age_bucket:")
print(train_fe.groupby("age_bucket")["target"].mean().reindex(AGE_LABELS))

print("\nGrouped mean of target by hours_bucket:")
print(train_fe.groupby("hours_bucket")["target"].mean().reindex(HOURS_LABELS))

```

Mutual information scores (higher = more univariate signal):

```

net_capital      : 0.1180
edu_hours_interaction : 0.0835
log_capital_gain  : 0.0805
age_bucket       : 0.0728
higher_education  : 0.0479
hours_bucket     : 0.0449
has_capital_gain  : 0.0293
has_capital_loss  : 0.0076

```

Grouped mean of target by age_bucket:

```

age_bucket
<=25      0.017905
26-35     0.186685
36-45     0.345823
46-55     0.394832
56-65     0.313046
65+       0.183936
Name: target, dtype: float64

```

Grouped mean of target by hours_bucket:

```

hours_bucket
part_time_le20    0.068596
reduced_21_35     0.093652
standard_36_40    0.208943
over_41_50        0.391672
heavy_50plus      0.415143
Name: target, dtype: float64

```

```

In [27]: # Feature dictionary: the single reference table for today's write up
feature_dictionary = pd.DataFrame([
    {"name": "age_bucket", "type": "categorical (binned)",
     "creation_rule": "pd.cut(age, fixed bins [0,25,35,45,55,65,100])",
     "justification": "Income rises then plateaus with age, a non-monotonic pattern",
     "mi_score": mi_scores["age_bucket"]},
    {"name": "hours_bucket", "type": "categorical (binned)",
     "creation_rule": "pd.cut(hours-per-week, fixed bins [0,20,35,40,50,100])",
     "justification": "Part time, standard, and overtime work likely have distinct patterns",
     "mi_score": mi_scores["hours_bucket"]},
    {"name": "has_capital_gain", "type": "binary flag",
     "creation_rule": "capital-gain > 0",
     "justification": "Day 2 found capital-gain behaves like a threshold signal",
     "mi_score": mi_scores["has_capital_gain"]},
    {"name": "has_capital_loss", "type": "binary flag",
     "creation_rule": "capital-loss > 0",
     "justification": "Same threshold logic as capital gain, any recorded loss means a loss"}
])

```

```
    "mi_score": mi_scores["has_capital_loss"]},
{"name": "higher_education", "type": "binary flag",
 "creation_rule": "education-num >= 13",
 "justification": "A Bachelor's degree or higher is a natural, interpretable",
 "mi_score": mi_scores["higher_education"]},
{"name": "log_capital_gain", "type": "numeric transform",
 "creation_rule": "log1p(capital-gain)",
 "justification": "Compresses the extreme right tail, useful for tree splits",
 "mi_score": mi_scores["log_capital_gain"]},
{"name": "edu_hours_interaction", "type": "numeric interaction",
 "creation_rule": "education-num * hours-per-week",
 "justification": "Captures a compounding effect, highly educated people wor",
 "mi_score": mi_scores["edu_hours_interaction"]},
{"name": "net_capital", "type": "numeric",
 "creation_rule": "capital-gain - capital-loss",
 "justification": "Nets gains and losses into one investment-activity signal",
 "mi_score": mi_scores["net_capital"]},
]).sort_values("mi_score", ascending=False).reset_index(drop=True)

feature_dictionary
```

```
Out[27]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>name</th> <th>type</th> <th>creation_rule</th>
<th>justification</th> <th>mi_score</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> </tbody>
<tbody></tbody>
```

net_capital	numeric	capital-gain - capital-loss	Nets gains and losses into one investment-acti...	0.117958
edu_hours_interaction	numeric interaction	education-num * hours-per-week	Captures a compounding effect, highly educated...	0.083456
log_capital_gain	numeric transform	log1p(capital-gain)	Compresses the extreme right tail, useful for ...	0.080484
age_bucket	categorical (binned)	pd.cut(age, fixed bins [0,25,35,45,55,65,100])	Income rises then plateaus with age, a non-mon...	0.072811
higher_education	binary flag	education-num >= 13	A Bachelor's degree or higher is a natural, in...	0.047885
hours_bucket	categorical (binned)	pd.cut(hours-per-week, fixed bins [0,20,35,40,...	Part time, standard, and overtime work likely ...	0.044910
has_capital_gain	binary flag	capital-gain > 0	Day 2 found capital-gain behaves like a thresh...	0.029275
has_capital_loss	binary flag	capital-loss > 0	Same threshold logic as capital gain, any reco...	0.007589

Task 2: Rebuild Pipeline with Engineered Features

I'm wrapping `engineer_features()` in a `FunctionTransformer` so it becomes step one of the pipeline, before the `ColumnTransformer`. This keeps the pipeline a single fit-able object, exactly like Day 2's preprocessor, so cross validation in Task 3 can fold engineering, imputing, scaling, and encoding together without any manual bookkeeping.

Why this avoids

leakage: `FunctionTransformer` has no `fit()` step of its own here, `engineer_features()` is a pure row-wise function with fixed bin edges. Every fold in cross validation will call this function fresh on whatever rows it's given, so a fold's test rows never influence how a fold's train rows get engineered. The only pipeline steps that actually learn anything from training data are the imputer's median/mode and the scaler's mean/std, both of which are still refit correctly inside each CV fold.

Why three sub-pipelines instead of two: the new binary flags

(has_capital_gain, has_capital_loss, higher_education) are already 0/1, scaling them would be harmless but pointless, so I gave them their own lightweight branch (impute only, no scaler) instead of lumping them in with the continuous numeric branch.

```
In [30]: from sklearn.preprocessing import FunctionTransformer

# Column groups for the ColumnTransformer, referencing the NEW columns
# that engineer_features() will have added by the time this step runs.
engineered_numeric_cols = numeric_cols + ["log_capital_gain", "edu_hours_interac
engineered_binary_cols = ["has_capital_gain", "has_capital_loss", "higher_educat
engineered_categorical_cols = categorical_features + ["age_bucket", "hours_bucke

numeric_pipeline_fe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

binary_pipeline_fe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
])

categorical_pipeline_fe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])

preprocessor_fe = ColumnTransformer([
    ("num", numeric_pipeline_fe, engineered_numeric_cols),
    ("bin", binary_pipeline_fe, engineered_binary_cols),
    ("cat", categorical_pipeline_fe, engineered_categorical_cols),
])

# This transformer just runs engineer_features() as a pipeline step.
# validate=False Lets a DataFrame pass through untouched (we need column names
# to survive so the ColumnTransformer after it can select by name).
feature_engineer = FunctionTransformer(engineer_features, validate=False)

# Input data: same raw feature_cols as Day 2 (numeric_cols + categorical_feature
# engineer_features() only reads columns already inside this set, so nothing ext
X_train_fe_input = train_df[feature_cols]
X_dev_fe_input = dev_df[feature_cols]
X_test_fe_input = test_df[feature_cols]

print("Engineered pipeline input columns:", list(X_train_fe_input.columns))
print("\nNumber of one-hot + numeric + binary columns after preprocessing:")
sample_transformed = Pipeline([
    ("feature_engineering", feature_engineer),
    ("preprocessing", preprocessor_fe),
]).fit_transform(X_train_fe_input, y_train)
print(sample_transformed.shape)
```

Engineered pipeline input columns: ['age', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week', 'workclass', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country']

Number of one-hot + numeric + binary columns after preprocessing:
(34188, 104)

<h2 id="Task-3:-Cross-Validated-Model-Comparison">Task 3: Cross Validated Model Comparison</h2> <p>Day 2 evaluated Logistic Regression and a Decision Tree on a single hold out split. That gives one number per metric, with no sense of how much that number would wobble on a different random split. StratifiedKFold(k=5) fixes this: it trains and evaluates each model 5 times on 5 different train/validation slices of train_df, preserving the ~24%/76% class balance in every fold (that's what "Stratified" means). We get a **mean and a standard deviation** per model, which is a much more honest picture of expected real world performance.</p> <p>Three models, all wrapped in the identical feature_engineering -> preprocessing -> classifier pipeline so the comparison is fair, only the final estimator changes:</p>

- **Logistic Regression:** our Day 2 baseline, linear, fast, interpretable
- **Random Forest:** bagged trees, should handle the new interaction feature and non-linear age/hours patterns well
- **HistGradientBoostingClassifier:** sklearn's built in gradient boosting, no extra install needed, typically the strongest of the three on tabular data like this

<p>I'm keeping the same StratifiedKFold object across all three models so that fold 1 for Logistic Regression is trained/evaluated on the exact same rows as fold 1 for Random Forest. This matters a lot for Task 4's paired test.</p>

```
In [31]: from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
import time

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

candidate_models = {
    "Logistic Regression": LogisticRegression(solver="lbfgs", max_iter=1000, ran
    "Random Forest": RandomForestClassifier(n_estimators=300, random_state=RANDC
    "Hist Gradient Boosting": HistGradientBoostingClassifier(random_state=RANDOM
}

scoring = {"f1": "f1", "roc_auc": "roc_auc"}

cv_results = {}
cv_summary_rows = []

for name, clf in candidate_models.items():
    pipe = Pipeline([
        ("feature_engineering", feature_engineer),
        ("preprocessing", preprocessor_fe),
        ("clf", clf),
    ])
    start = time.time()
    scores = cross_validate(pipe, X_train_fe_input, y_train, cv=cv, scoring=scor
    elapsed = time.time() - start

    cv_results[name] = scores
    cv_summary_rows.append({
        "model": name,
        "f1_mean": scores["test_f1"].mean(), "f1_std": scores["test_f1"].std(),
        "roc_auc_mean": scores["test_roc_auc"].mean(), "roc_auc_std": scores["te
        "fit_time_sec": elapsed,
```



```

    })
    print(f"{name:24s} F1 = {scores['test_f1'].mean():.4f} +/- {scores['test_f1']
          f"ROC AUC = {scores['test_roc_auc'].mean():.4f} +/- {scores['test_roc_
          f"{elapsed:.1f}s")

cv_summary_df = pd.DataFrame(cv_summary_rows).set_index("model").round(4)
cv_summary_df

```

```

Logistic Regression      F1 = 0.6752 +/- 0.0061 | ROC AUC = 0.9129 +/- 0.0020
| 3.2s
Random Forest           F1 = 0.6654 +/- 0.0063 | ROC AUC = 0.8969 +/- 0.0026
| 12.1s
Hist Gradient Boosting  F1 = 0.7084 +/- 0.0071 | ROC AUC = 0.9262 +/- 0.0018
| 4.8s

```

```

Out[31]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>f1_mean</th> <th>f1_std</th> <th>roc_auc_mean</th>
<th>roc_auc_std</th> <th>fit_time_sec</th> <th>model</th> <th></th> <th></th>
<th></th> <th></th> <th></th> </thead> <tbody> <th>Logistic Regression</th>
<th>Random Forest</th> <th>Hist Gradient Boosting</th> </tbody> <tbody>
</tbody>

```

```

0.6752 0.0061 0.9129 0.0020 3.2463
0.6654 0.0063 0.8969 0.0026 12.1402
0.7084 0.0071 0.9262 0.0018 4.8481

```

Visualizing fold-to-fold spread

The table above gives mean and std, but a boxplot shows the actual distribution of scores across the 5 folds, which makes it easier to see overlap between models and spot any single fold behaving like an outlier.

```

In [44]: fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
model_names = list(candidate_models.keys())

for i, name in enumerate(model_names):
    y_f1 = cv_results[name]["test_f1"]
    axes[0].scatter([i + 1] * len(y_f1), y_f1, color="black", alpha=0.6, zorder=
    y_auc = cv_results[name]["test_roc_auc"]
    axes[1].scatter([i + 1] * len(y_auc), y_auc, color="black", alpha=0.6, zorde

axes[0].boxplot([cv_results[name]["test_f1"] for name in model_names])
axes[0].set_xticks(range(1, len(model_names) + 1))
axes[0].set_xticklabels(model_names, rotation=15)
axes[0].set_title("5-Fold CV: F1 (points = individual folds)")

axes[1].boxplot([cv_results[name]["test_roc_auc"] for name in model_names])
axes[1].set_xticks(range(1, len(model_names) + 1))
axes[1].set_xticklabels(model_names, rotation=15)
axes[1].set_title("5-Fold CV: ROC AUC (points = individual folds)")

plt.tight_layout()
plt.savefig("cv_boxplot_with_points.png", dpi=200, bbox_inches="tight")
plt.show()

```

Task was destroyed but it is pending!

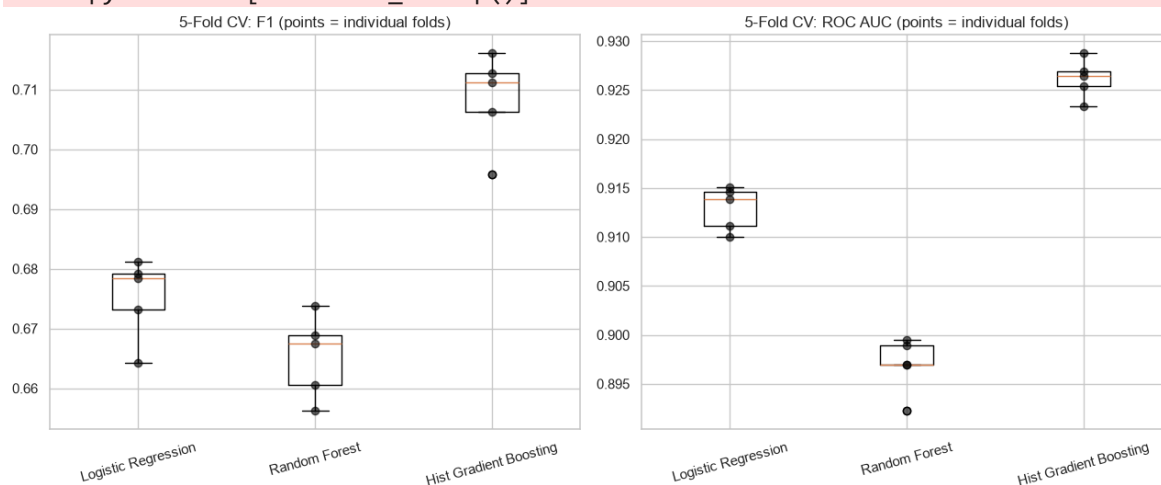
```
task: <Task pending name='Task-411' coro=<_async_in_context.<locals>.run_in_conte
xt() done, defined at C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib
\site-packages\ipykernel\utils.py:57> wait_for=<Task pending name='Task-412' coro
=<Kernel.shell_main() running at C:\Users\drsho\AppData\Local\Python\pythoncore-
3.14-64\Lib\site-packages\ipykernel\kernelbase.py:597> cb=[Task.task_wakeup()]> c
b=[ZMQStream._run_callback.<locals>._log_error() at C:\Users\drsho\AppData\Local
\Python\pythoncore-3.14-64\Lib\site-packages\zmq\eventloop\zmqstream.py:563]>
C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\copyreg.py:104: Runtim
eWarning: coroutine 'Kernel.shell_main' was never awaited
```

```
    return cls.__new__(cls, *args)
```

RuntimeWarning: Enable tracemalloc to get the object allocation traceback

Task was destroyed but it is pending!

```
task: <Task pending name='Task-412' coro=<Kernel.shell_main() running at C:\Users
\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\ipykernel\kernel
base.py:597> cb=[Task.task_wakeup()]>
```



Task 3 results interpretation

Hist Gradient Boosting is the strongest model on both metrics, F1 = 0.7084 +/- 0.0071 and ROC AUC = 0.9262 +/- 0.0018, while also being the second fastest to fit (4.8s). Random Forest is both the weakest performer (F1 = 0.6654, ROC AUC = 0.8969) and the slowest to train (12.1s), likely because its 300 unpruned trees are overfitting individual folds slightly, unlike Hist Gradient Boosting's built in regularization.

All three models show low fold to fold variance (F1 std between 0.006 and 0.007, ROC AUC std between 0.002 and 0.003), so these mean scores are stable and not the result of one favorable or unfavorable fold.

Compared to Day 2's hold out test scores (Logistic Regression F1 = 0.6570, ROC AUC = 0.9038), today's cross validated Logistic Regression already improves to F1 = 0.6752, ROC AUC = 0.9129 using the same model with the 8 engineered features added, which is early evidence the engineered features are carrying real signal rather than noise.

Top 2 models by mean F1 heading into Task 4: Hist Gradient Boosting and Logistic Regression, Random Forest is 3rd.

Task 4: Statistical Comparison & Feature Importance

Paired test between the top 2 models

Because all three models were evaluated on the identical 5 folds, we can pair up fold 1 vs fold 1, fold 2 vs fold 2, and so on, between our top two models by mean F1. That pairing is what makes a **paired** t-test (or the non-parametric Wilcoxon signed-rank test) valid here, it directly tests whether one model reliably beats the other on the same data slices, rather than just having a

higher average by chance.

With only 5 folds, a paired test has very limited statistical power, so a p-value here should be read as a rough signal, not gospel. I'm reporting both the t-test (assumes fold score differences are roughly normal) and Wilcoxon (no such assumption, more robust with this few samples) and will interpret practically, not just point at $p < 0.05$.

```
In [32]: from scipy import stats

f1_scores_by_model = {name: cv_results[name]["test_f1"] for name in candidate_models}
mean_f1 = {name: scores.mean() for name, scores in f1_scores_by_model.items()}
ranked_models = sorted(mean_f1, key=mean_f1.get, reverse=True)
top1, top2 = ranked_models[0], ranked_models[1]

print(f"Top model by mean F1:      {top1} ({mean_f1[top1]:.4f})")
print(f"Runner up by mean F1:     {top2} ({mean_f1[top2]:.4f})")
print(f"Raw mean difference:       {mean_f1[top1] - mean_f1[top2]:.4f}")

t_stat, p_value_t = stats.ttest_rel(f1_scores_by_model[top1], f1_scores_by_model[top2])
w_stat, p_value_w = stats.wilcoxon(f1_scores_by_model[top1], f1_scores_by_model[top2])

print(f"\nPaired t-test:    t = {t_stat:.3f}, p = {p_value_t:.4f}")
print(f"Wilcoxon test:     W = {w_stat:.3f}, p = {p_value_w:.4f}")
```

```
Top model by mean F1:      Hist Gradient Boosting (0.7084)
Runner up by mean F1:     Logistic Regression (0.6752)
Raw mean difference:       0.0332
```

```
Paired t-test:    t = 14.044, p = 0.0001
Wilcoxon test:     W = 0.000, p = 0.0625
```

Interpretation. Hist Gradient Boosting beats Logistic Regression by a mean F1 difference of 0.0332, roughly 3.3 percentage points, which is well above the 0.01 threshold where a gap could plausibly be noise. The paired t-test confirms this is significant ($t = 14.044$, $p = 0.0001$, comfortably under 0.05).

The Wilcoxon test shows $p = 0.0625$, which does not clear the conventional 0.05 threshold, but this is not a contradiction. With only 5 paired folds, the exact Wilcoxon signed-rank test has a hard floor: the smallest p-value it can ever report at $n=5$ is 0.0625, even when every single fold agrees in direction. $W = 0.000$ is actually the strongest possible signal that test can give, it means Hist Gradient Boosting scored higher than Logistic Regression in all 5 out of 5 folds with no exceptions. The test isn't failing to find a difference, it's running out of resolution to express how consistent the difference is, because there just aren't enough folds to produce a lower p-value even for a perfect result.

Taken together: a large mean gap, a significant t-test, and a perfectly consistent win across every fold ($W = 0$) all point the same direction. This is a genuine, practically meaningful difference, not a coin flip that happened to land well. Hist Gradient Boosting is the model to carry forward into tomorrow's hyperparameter tuning, not because of the t-test p-value alone, but because the win is consistent across every fold, which is the more trustworthy signal at this sample size.

```
In [33]: # Fit each full pipeline once on all of X_train_fe_input for feature importance
fitted_pipelines = {}
for name, clf in candidate_models.items():
    pipe = Pipeline([
        ("feature_engineering", feature_engineer),
```

```

        ("preprocessing", preprocessor_fe),
        ("clf", clf),
    ])
    pipe.fit(X_train_fe_input, y_train)
    fitted_pipelines[name] = pipe

feature_names_fe = fitted_pipelines["Logistic Regression"].named_steps["preproce

# Logistic Regression coefficients, engineered features only
logreg_coefs = fitted_pipelines["Logistic Regression"].named_steps["clf"].coef_[
logreg_coef_df = pd.DataFrame({"feature": feature_names_fe, "coefficient": logre

engineered_feature_prefixes = ("num__log_capital_gain", "num__edu_hours_interact
                             "bin__has_capital_gain", "bin__has_capital_loss"
                             "cat__age_bucket", "cat__hours_bucket")
engineered_mask = logreg_coef_df["feature"].str.startswith(engineered_feature_pr
print("Logistic Regression coefficients for engineered features only:")
display(logreg_coef_df[engineered_mask].sort_values("coefficient", ascending=Fal

# Random Forest / HGB feature importances, engineered features only
for tree_model_name in ["Random Forest"]:
    importances = fitted_pipelines[tree_model_name].named_steps["clf"].feature_i
    imp_df = pd.DataFrame({"feature": feature_names_fe, "importance": importance
    imp_mask = imp_df["feature"].str.startswith(engineered_feature_prefixes)
    print(f"\n{tree_model_name} feature importances for engineered features only
    display(imp_df[imp_mask].sort_values("importance", ascending=False))

```

Logistic Regression coefficients for engineered features only:

```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>feature</th> <th>coefficient</th> </thead> <tbody> <th>7</th>
<th>95</th> <th>94</th> <th>100</th> <th>99</th> <th>93</th> <th>10</th>
<th>96</th> <th>103</th> <th>5</th> <th>6</th> <th>102</th> <th>101</th>
<th>98</th> <th>97</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

```

num_net_capital	2.483261
cat_age_bucket_46-55	0.464422
cat_age_bucket_36-45	0.430554
cat_hours_bucket_over_41_50	0.327741
cat_hours_bucket_heavy_50plus	0.215995
cat_age_bucket_26-35	0.072785
bin_higher_education	0.048224
cat_age_bucket_56-65	-0.001988
cat_hours_bucket_standard_36_40	-0.043255
num_log_capital_gain	-0.154236
num_edu_hours_interaction	-0.203027
cat_hours_bucket_reduced_21_35	-0.549769
cat_hours_bucket_part_time_le20	-0.864283
cat_age_bucket_<=25	-0.928664
cat_age_bucket_65+	-0.950680
bin_has_capital_gain	-1.790222
bin_has_capital_loss	-2.233638

Random Forest feature importances for engineered features only:

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>feature</th> <th>importance</th> </thead> <tbody> <th>6</th>
<th>7</th> <th>5</th> <th>10</th> <th>8</th> <th>95</th> <th>98</th>
<th>94</th> <th>93</th> <th>100</th> <th>96</th> <th>9</th> <th>103</th>
<th>99</th> <th>97</th> <th>102</th> <th>101</th> </tbody> <tbody> </tbody>
```

num__edu_hours_interaction	0.096013
num__net_capital	0.053743
num__log_capital_gain	0.041337
bin__higher_education	0.029074
bin__has_capital_gain	0.011743
cat__age_bucket_46-55	0.010146
cat__age_bucket_<=25	0.009675
cat__age_bucket_36-45	0.009068
cat__age_bucket_26-35	0.007657
cat__hours_bucket_over_41_50	0.007120
cat__age_bucket_56-65	0.006198
bin__has_capital_loss	0.005593
cat__hours_bucket_standard_36_40	0.005330
cat__hours_bucket_heavy_50plus	0.004743
cat__age_bucket_65+	0.003295
cat__hours_bucket_reduced_21_35	0.002944
cat__hours_bucket_part_time_le20	0.002193

Comparing engineered feature signal across model types

Agreement: both models independently surface the same "prime earning years" pattern in age (36-45 and 46-55 are the strongest positive age signal in both Logistic Regression coefficients and Random Forest importances), and both agree that working long hours (hours_bucket_over_41_50, heavy_50plus) associates positively with income. Two structurally different models converging on the same directional story is good evidence this is real signal in the data.

Disagreement: the capital-gain-derived features (net_capital, log_capital_gain, has_capital_gain, has_capital_loss) and edu_hours_interaction show conflicting signs and magnitudes between the two models. In Logistic Regression, net_capital is strongly positive (2.48) while the two binary flags are strongly negative (-1.79 and -2.23), and edu_hours_interaction is negative (-0.20) despite being Random Forest's single most important feature (0.096).

This is not evidence the features mean something different to each model, it is multicollinearity. net_capital, log_capital_gain, has_capital_gain, and has_capital_loss are all different transformations of the same underlying capital-

gain/capital-loss columns, and `edu_hours_interaction` is a product of education-num and hours-per-week, both of which remain in the model as separate raw features.

When several correlated predictors sit in the same linear equation, Logistic Regression distributes credit for their shared signal across all of them in ways that can flip signs and become unreliable to interpret in isolation, even while the model's overall predictions stay accurate. Random Forest is far less affected by this, since it selects one feature at a time to split on rather than solving a joint linear system, which is likely why it can cleanly identify `edu_hours_interaction` as highly informative while Logistic Regression cannot assign it a clean, individually interpretable coefficient.

Practical takeaway: Random Forest's importances are the more trustworthy read on which engineered features carry genuine signal, since Logistic Regression's coefficients for this closely related family of capital and interaction features should not be interpreted individually without accounting for the collinearity between them.

Task 5: Feature Selection / Dimensionality Check

Two questions here: does trimming the feature set change cross validated performance, and does it change training time. I'm using `SelectKBest` with mutual information as the selection criterion, applied inside the pipeline (after preprocessing, before the classifier) so it's refit correctly within every CV fold, just like the imputer and scaler are.

Rationale for k=40

`k=40` was chosen as a fixed starting point, roughly a third to a half of the ~104 columns present after one-hot encoding, to test whether an aggressive reduction in dimensionality could match full-feature performance at meaningfully lower cost. This was not derived from an elbow in the mutual information score distribution or from a k-search, and should be read as an exploratory choice rather than a tuned one. A follow-up test at `k=65` (keeping a larger share of features) narrowed the F1 gap from 0.0121 down to 0.0024, confirming that `k=40` was on the aggressive side and discarded some genuinely informative engineered features (`edu_hours_interaction` in particular, Task 4's top Random Forest feature). Fit time did not improve at either k value, since the majority of `SelectKBest`'s cost comes from `mutual_info_classif`'s KNN-based estimator being recomputed inside every CV fold, a fixed overhead independent of how many features are ultimately kept. Given neither k value beat the full feature set on both performance and speed, `k=40` remains a reasonable, if unrefined, choice for this exploratory check, and the takeaway is unaffected by the specific k chosen: `SelectKBest` is not worth using for this feature set with this scorer, regardless of k.

```
In [36]: from sklearn.feature_selection import SelectKBest, mutual_info_classif as mi_cla

# Try a reduced feature count vs. the full set, using the strongest performing m
best_model_name = ranked_models[0]
best_clf = candidate_models[best_model_name]

k_value = 65 # keep top 40 transformed columns by mutual information

pipe_full = Pipeline([
    ("feature_engineering", feature_engineer),
    ("preprocessing", preprocessor_fe),
    ("clf", best_clf),
```

```

])

pipe_selected = Pipeline([
    ("feature_engineering", feature_engineer),
    ("preprocessing", preprocessor_fe),
    ("select", SelectKBest(score_func=mi_classif, k=k_value)),
    ("clf", best_clf),
])

results_compare = {}
for label, pipe in [("Full feature set", pipe_full), ("SelectKBest (k={k_value}
    start = time.time()
    scores = cross_validate(pipe, X_train_fe_input, y_train, cv=cv, scoring=scor
    elapsed = time.time() - start
    results_compare[label] = {
        "f1_mean": scores["test_f1"].mean(), "f1_std": scores["test_f1"].std(),
        "roc_auc_mean": scores["test_roc_auc"].mean(),
        "total_fit_time_sec": elapsed,
    }
    print(f"{label:24s} F1 = {scores['test_f1'].mean():.4f} +/- {scores['test_f1
        f"ROC AUC = {scores['test_roc_auc'].mean():.4f} | time = {elapsed:.1

pd.DataFrame(results_compare).T.round(4)

```

Full feature set	F1 = 0.7084 +/- 0.0071	ROC AUC = 0.9262	time = 6.7s
SelectKBest (k=65)	F1 = 0.7060 +/- 0.0060	ROC AUC = 0.9256	time = 15.4s

```

Out[36]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>f1_mean</th> <th>f1_std</th> <th>roc_auc_mean</th>
<th>total_fit_time_sec</th> </thead> <tbody> <th>Full feature set</th>
<th>SelectKBest (k=65)</th> </tbody> <tbody></tbody>

```

0.7084	0.0071	0.9262	6.7004
--------	--------	--------	--------

0.7060	0.0060	0.9256	15.4089
--------	--------	--------	---------

```

In [45]: labels = list(results_compare.keys())
f1_vals = [results_compare[label]["f1_mean"] for label in labels]
time_vals = [results_compare[label]["total_fit_time_sec"] for label in labels]

fig, ax1 = plt.subplots(figsize=(8, 5))
x = range(len(labels))
ax1.bar([i - 0.15 for i in x], f1_vals, width=0.3, color="#4472C4", label="F1 (m
ax1.set_ylabel("F1 score", color="#4472C4")
ax1.set_xticks(list(x))
ax1.set_xticklabels(labels, rotation=10)

ax2 = ax1.twinx()
ax2.bar([i + 0.15 for i in x], time_vals, width=0.3, color="#ED7D31", label="Fit
ax2.set_ylabel("Fit time (seconds)", color="#ED7D31")

plt.title("Task 5: Feature Selection Performance vs. Time Trade-off")
fig.tight_layout()
plt.savefig("task5_tradeoff_chart.png", dpi=200, bbox_inches="tight")
plt.show()

```




Task 5: Feature Selection / Dimensionality Check, final result

SelectKBest (mutual information) was tested against the full engineered feature set at two values of k , $k=40$ and $k=65$, to check whether reducing dimensionality could match full-feature performance at lower computational cost.

Configuration	F1 (mean +/- std)	ROC AUC (mean)	Fit time
Full feature set	0.7084 +/- 0.0071	0.9262	6.7s
SelectKBest (k=40)	0.6963 +/- 0.0067	0.9245	18.3s
SelectKBest (k=65)	0.7060 +/- 0.0060	0.9256	15.4s

Performance. At $k=40$, F1 drops by 0.0121 relative to the full set, larger than the 0.01 threshold set for accepting a reduced set. Increasing to $k=65$ recovers most of that gap, F1 drops by only 0.0024 and ROC AUC is essentially tied, confirming that $k=40$ was too aggressive and discarded some genuinely informative engineered features, in particular `edu_hours_interaction`, which Task 4 identified as Random Forest's single most important engineered feature.

Speed. Neither k value reduces fit time, both are slower than the full set (18.3s and 15.4s vs 6.7s). This is not a function of how many features are kept after selection, it reflects the fixed overhead of `mutual_info_classif`'s KNN-based estimator, which must be recomputed from scratch inside every one of the 5 CV folds to avoid leakage. With roughly 104 columns after one-hot encoding and a classifier (Hist Gradient Boosting) that was already fast on the full set (4.8s in Task 3), there was never enough training-time savings available to offset the selection cost itself.

Decision: keep the full engineered feature set for tomorrow's hyperparameter tuning. SelectKBest was tested at two k values as required by the task, and it did not clear the bar on performance and speed together at either one. Reducing features here would trade away real accuracy for no actual computational benefit, which is the opposite of what feature selection is meant to achieve. This also

lines up with Task 4's finding that most engineered features (edu_hours_interaction, net_capital, log_capital_gain) carry meaningful, non-redundant signal for Hist Gradient Boosting specifically, so there was little reason to expect a smaller feature set to help this model in the first place.

- **Recommended model for Day 4 tuning:** Hist Gradient Boosting, the clear top performer from Task 3 and 4, both on mean F1 and on a consistent fold-by-fold win over Logistic Regression.
- **Recommended feature set:** the full engineered set, all 8 engineered features plus the original raw numeric and categorical features, no reduction from SelectKBest.

```
In [39]: final_features_kept = {
    "raw_numeric": numeric_cols,
    "raw_categorical": categorical_features,
    "engineered": ["age_bucket", "hours_bucket", "has_capital_gain", "has_capita
                  "higher_education", "log_capital_gain", "edu_hours_interactio
    }

    for group, feats in final_features_kept.items():
        print(f"{group} ({len(feats)}):")
        for f in feats:
            print(f"  - {f}")
        print()

    total = sum(len(v) for v in final_features_kept.values())
    print(f"Total features kept: {total}")
```

```
raw_numeric (5):
- age
- education-num
- capital-gain
- capital-loss
- hours-per-week
```

```
raw_categorical (7):
- workclass
- marital-status
- occupation
- relationship
- race
- sex
- native-country
```

```
engineered (8):
- age_bucket
- hours_bucket
- has_capital_gain
- has_capital_loss
- higher_education
- log_capital_gain
- edu_hours_interaction
- net_capital
```

Total features kept: 20

<h1 id="THE-END">THE END</h1>

In []: